Innopolis University

Big Data — Final Project, 2026

Detecting IoT Malware at Network Scale

An end-to-end Hadoop / Spark pipeline over the Aposemat IoT-23 corpus

Team: Team 30
Repository: Dmitry057/big-data-big-final-group-case-study-project
Local path: `/home/team30/project30`
Cluster host: `hadoop-01.uni.innopolis.ru`

Ivan Ilyichev Data Engineer
Kirill Shumsky ML Specialist
Nikita Zagainov Data Scientist
Dmitry Tetkin Tester & Technical Writer

May 9, 2026

Contents

1 Introduction **2**
1.1 Business objectives . . . 2
1.2 Pipeline at a glance . . . 2

2 Data Description **2**
2.1 Source . . . 2
2.2 Data characteristics . . . 3
2.3 Architecture of the data pipeline . . . 3
2.4 Per-stage input / output . . . 4

3 Data Preparation **4**
3.1 Relational model and ER diagram . . . 4
3.2 Sample records . . . 5
3.3 Hive warehouse . . . 5
3.4 Storage format benchmark . . . 5

4 Data Analysis **6**
4.1 Insight 1 — Label distribution . . . 6
4.2 Insight 2 — Protocol mix per label . . . 7
4.3 Insight 3 — Average bytes per connection . . . 8
4.4 Insight 4 — Per-capture maliciousness ratio . . . 9
4.5 Insight 5 — Connection state per label . . . 9
4.6 Insight 6 — Top targeted ports . . . 10
4.7 Insight 7 — Average packets per connection . . . 11

5 ML Modeling **12**
5.1 Feature extraction and preprocessing . . . 12
5.2 Models and hyperparameter tuning . . . 12
5.3 Evaluation . . . 12

6 Data Presentation **13**
6.1 Dashboard description . . . 13
6.2 Findings . . . 13

7 Conclusion **14**
7.1 Summary . . . 14
7.2 Reflections on own work . . . 14
7.3 Challenges and difficulties . . . 14
7.4 Recommendations . . . 14
7.5 Table of contributions . . . 15

1 Introduction

1.1 Business objectives

Internet-of-Things devices have become a vast and largely unmonitored attack surface. Compromised cameras, routers and digital video recorders are weaponised into botnets such as *Mirai*, *Hide-and-Seek* and *Torii*, generating distributed denial-of-service traffic, port scans and command-and-control flows that share a network with legitimate device traffic. The economic and operational damage of these botnets motivates the deployment of automated network-traffic classification at the scale of an entire ISP or data centre.

The business objective of this project is to demonstrate, on a representative real-world dataset, that:

1. Raw Zeek connection logs from heterogeneous IoT captures can be ingested, stored and queried at the scale of tens of millions of rows using a standard Hadoop/Spark stack.
2. A small set of statistical insights derived from the data can already separate benign traffic from several distinct attack behaviours.
3. A distributed multiclass classifier trained on the same warehouse can flag malicious connections with very high accuracy, providing a practical first-line filter for a security operations team.

The deliverable is a fully reproducible pipeline (a single `main.sh` that re-creates every artefact from the original CSV input), a public Apache Superset dashboard, and this report.

1.2 Pipeline at a glance

The project is organised in four reproducible stages, executed end-to-end by `main.sh`:

Stage	Script	Output
1 — Ingestion	`stage1.sh`	PostgreSQL DB + HDFS Parquet warehouse
2 — Storage & EDA	`stage2.sh`	Hive partitioned tables + 7 EDA result tables
3 — Predictive analytics	`stage3.sh`	Trained Spark ML models + held-out predictions
4 — Dashboard	`stage4.sh`	Apache Superset dashboard refreshed

Table 1: The four stages of the pipeline.

2 Data Description

2.1 Source

The dataset is a hand-picked subset of the **Aposemat IoT-23** corpus (Stratosphere Lab, Czech Technical University), consisting of twelve labelled captures of real IoT-device network traffic recorded over several years. Each capture contains a Zeek `conn.log` file describing every observed network connection, with both binary (`Benign`/`Malicious`) and detailed multi-class labels for the malicious flows.

2.2 Data characteristics

Property	Value
Total connections (rows)	25,011,247
Distinct captures	12
Per-row features (raw Zeek fields)	23
Target classes	7
Format on disk (cluster)	Parquet + Snappy
Approx. raw CSV size	∼737 MB

Table 2: Volume and shape of the working dataset.

The seven label values are: `Benign`, `Malicious`, `Malicious DDoS`, `Malicious PartOfAHorizontalPortScan`, `Malicious C&C`, `Malicious Attack`, `Malicious FileDownload`.

2.3 Architecture of the data pipeline

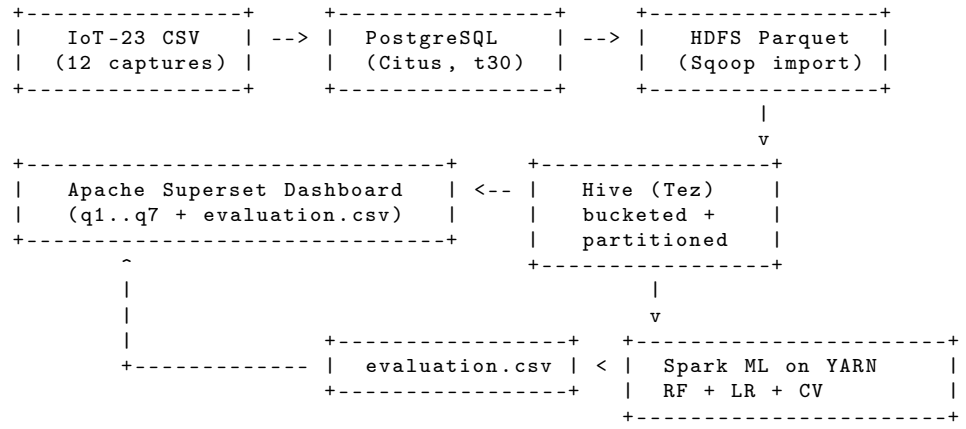

Figure 1: End-to-end pipeline: from raw CSV captures through PostgreSQL and HDFS to a Hive warehouse, distributed Spark ML and a Superset dashboard.

2.4 Per-stage input / output

Stage	Input	Output
Stage 1	12 IoT-23 capture CSV files	PostgreSQL tables `captures`, `network_connections`; HDFS Parquet/Snappy warehouse at `project/warehouse`
Stage 2	Sqoop Parquet warehouse on HDFS	Hive database `team30_projectdb` with bucketed/partitioned tables; 7 result tables `q1_results`...`q7_results`; CSV exports in `output/`
Stage 3	Hive table `network_connections_part` (label-partitioned)	Trained models `models/model1` (RF), `models/model2` (LR); predictions `output/modelN_predictions.csv`; metrics `output/evaluation.csv`
Stage 4	Hive result tables + `evaluation.csv`	Public Apache Superset dashboard

Table 3: Inputs and outputs of each pipeline stage.

3 Data Preparation

3.1 Relational model and ER diagram

The relational model in PostgreSQL consists of two tables connected by a single foreign key. `captures` is the dimensional reference for the twelve IoT-23 captures; `network_connections` is the fact-style table that holds every parsed Zeek connection record.

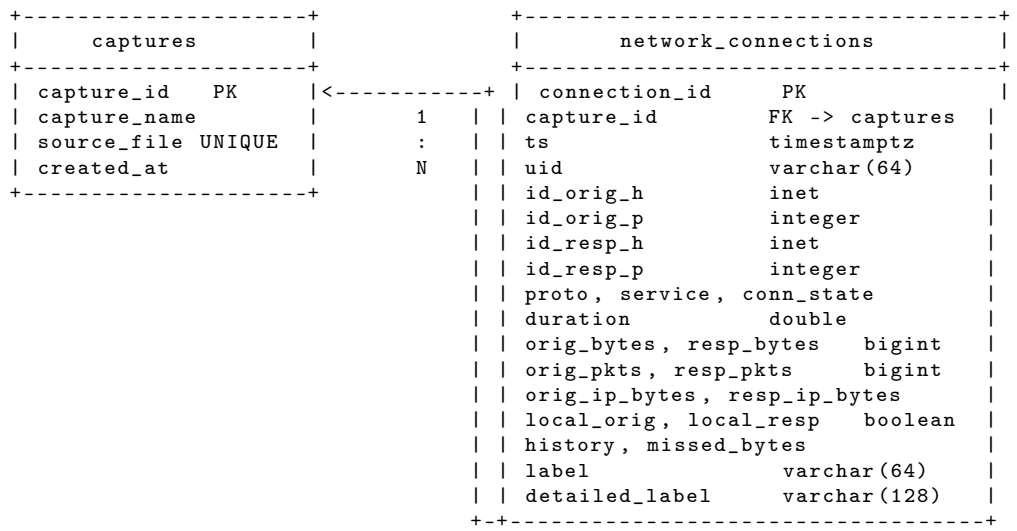

Figure 2: ER diagram. `captures` (1) — (N) `network_connections`.

A temporary staging table `stg_network_connections` is used to ingest the raw text via PostgreSQL `COPY`, after which a typed transformation insert (regex-cleaned integers, `to_timestamp`, `inet` casts) populates the canonical table.

3.2 Sample records

A sample of two rows from `network_connections` (truncated for width):

cap	ts	id_orig_h	proto	conn_state	orig_bytes	resp_bytes	orig_pkts	label
1	2018-12-21 14:02:11	192.168.1.132	tcp	S0	0	0	1	Malicious
9	2019-04-30 09:15:48	192.168.100.103	udp	SF	1024	64	4	Benign

Table 4: Two example records from the canonical `network_connections` table.

3.3 Hive warehouse

The Hive database `team30_projectdb` contains two physical layouts optimised for different access patterns:

- `captures_bucketed` — `CLUSTERED BY (capture_id) INTO 4 BUCKETS`, Parquet, used as the join dimension table.
- `network_connections_part` — `PARTITIONED BY (label)`, Parquet. Because every analytical query and every ML feature scan filters by label, partitioning at this column lets the Tez engine prune to a fraction of the data on every access.

The warehouse is materialised by `sql/db.hql`, run on the Tez engine with dynamic partitioning enabled:

```
SET hive.execution.engine=tez;
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
SET hive.enforce.bucketing=true;

INSERT OVERWRITE TABLE network_connections_part PARTITION (label)
SELECT connection_id, capture_id, CAST(ts AS TIMESTAMP), uid,
       id_orig_h, id_orig_p, id_resp_h, id_resp_p,
       proto, service, duration, orig_bytes, resp_bytes,
       conn_state, local_orig, local_resp, missed_bytes, history,
       orig_pkts, orig_ip_bytes, resp_pkts, resp_ip_bytes,
       tunnel_parents, detailed_label, label
FROM network_connections_ext;
```

3.4 Storage format benchmark

As an additional Stage-1 deliverable, four format/codec combinations were benchmarked on the same dataset (three runs each):

Format	**Codec**	**Runs**	**Avg write (s)**	**Avg full scan (s)**	**Avg analytic (s)**
parquet	snappy	3	17.7	6.76	9.29
parquet	gzip	3	18.0	6.64	9.56
avro	snappy	3	17.0	9.32	10.50
avro	bzip2	3	19.0	8.36	11.01

Table 5: Storage format benchmark. **Parquet + Snappy** was selected for the production warehouse based on the best balance of write and scan times.

4 Data Analysis

The seven HiveQL queries in `sql/q1.hql` . . . `sql/q7.hql` run on Tez against `network_connections_part`. Each query materialises a Parquet result table and exports a CSV, which the dashboard charts directly. The seven insights below correspond to the seven dashboard sections.

4.1 Insight 1 — Label distribution

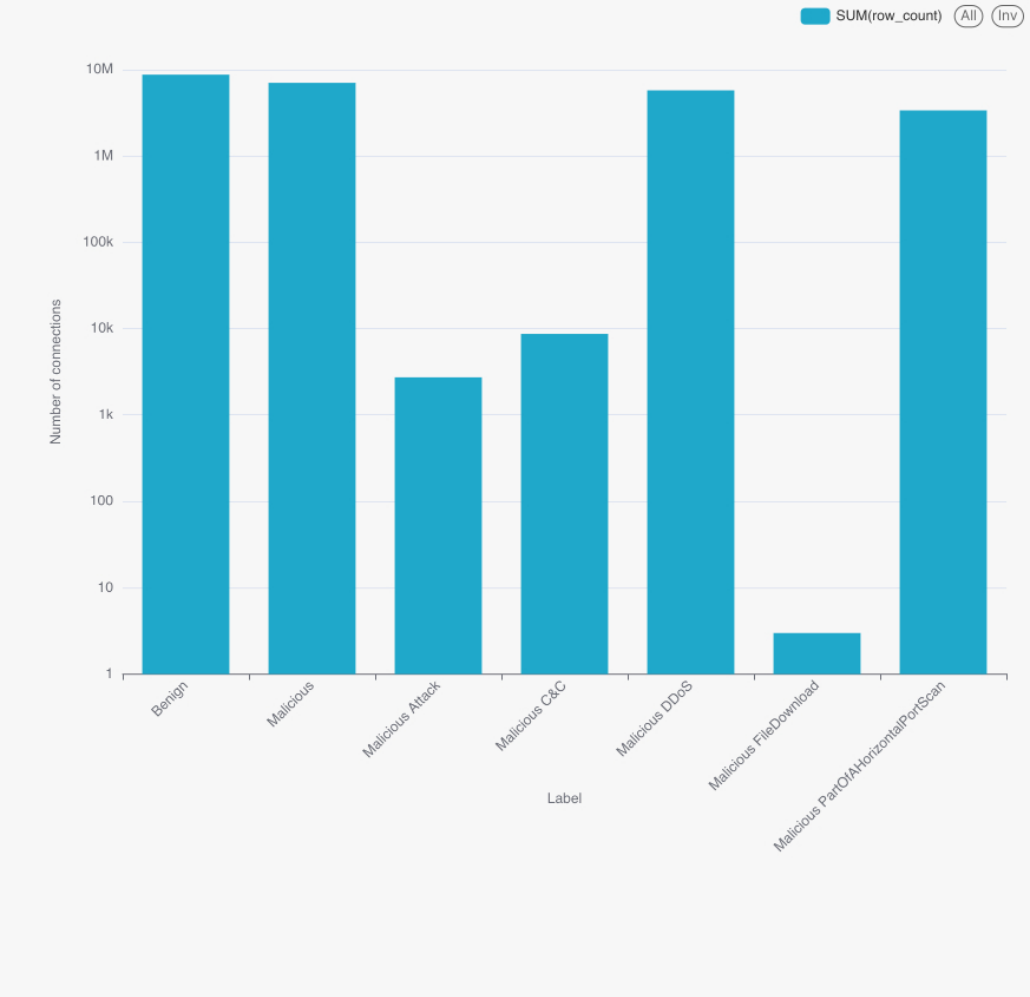

Figure 3: Class shares of `network_connections_part`.

Label	Rows	Share
Benign	8,780,158	35.11%
Malicious	7,055,007	28.21%
Malicious DDoS	5,778,154	23.10%
Malicious PartOfAHorizontalPortScan	3,386,241	13.54%
Malicious C&C	8,685	0.035%
Malicious Attack	2,755	0.011%
Malicious FileDownload	3	0.000%

Table 6: Label distribution.

Interpretation. The corpus is neither balanced nor degenerate. Four classes (Benign, Malicious, DDoS, port-scan) account for $> 99.9\%$ of the data, yet three rare attack types remain in the mix. This is the key reason the modelling task is treated as multiclass rather than binary — a binary model would silently absorb the rare classes into a single `Malicious` bucket and lose useful diagnostic signal.

4.2 Insight 2 — Protocol mix per label

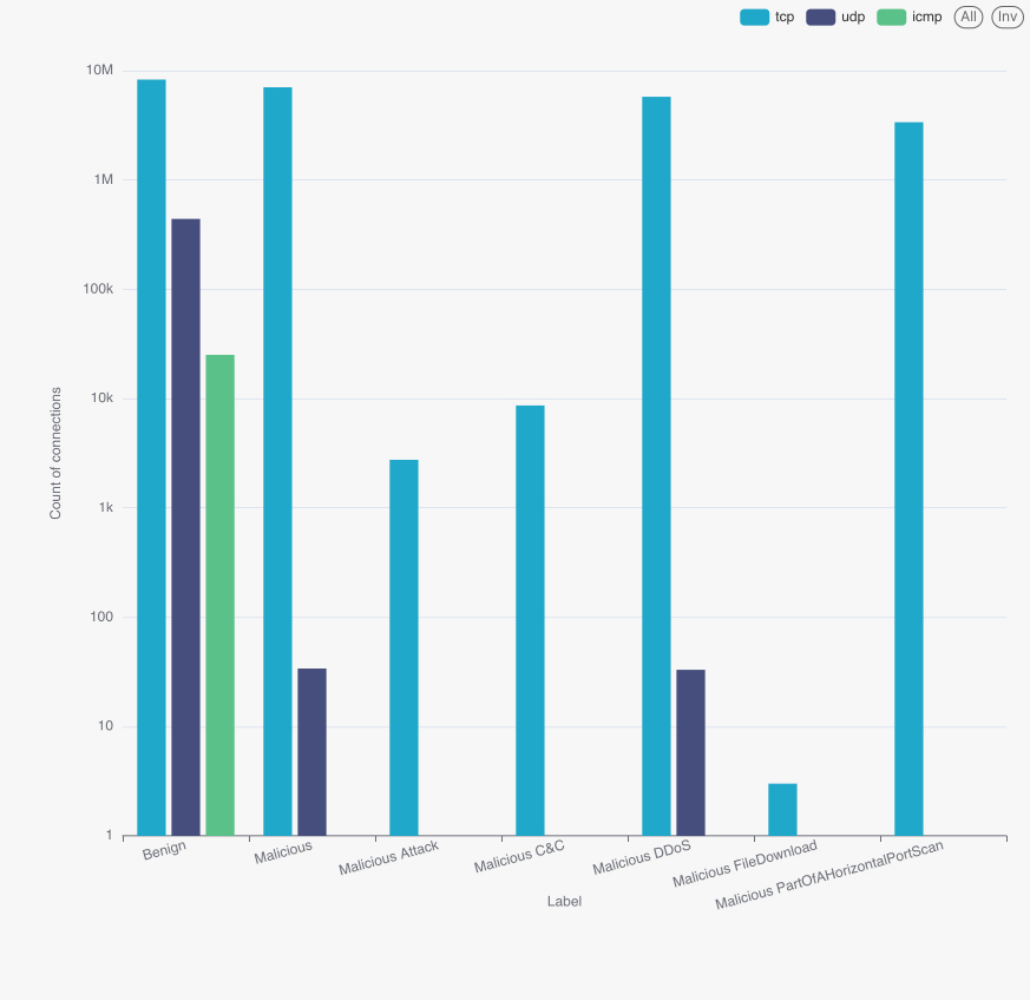

Figure 4: Protocol usage per label. UDP is essentially benign DNS; almost every malicious connection is TCP.

Interpretation. Apart from a tiny tail (34 UDP rows in the generic `Malicious` class, one UDP DDoS, etc.), every malicious connection in this corpus is TCP. ICMP appears only in benign baselines. The `proto` field alone is therefore not a useful discriminator inside the malicious population — but it is a hard prior against UDP traffic being malicious.

4.3 Insight 3 — Average bytes per connection

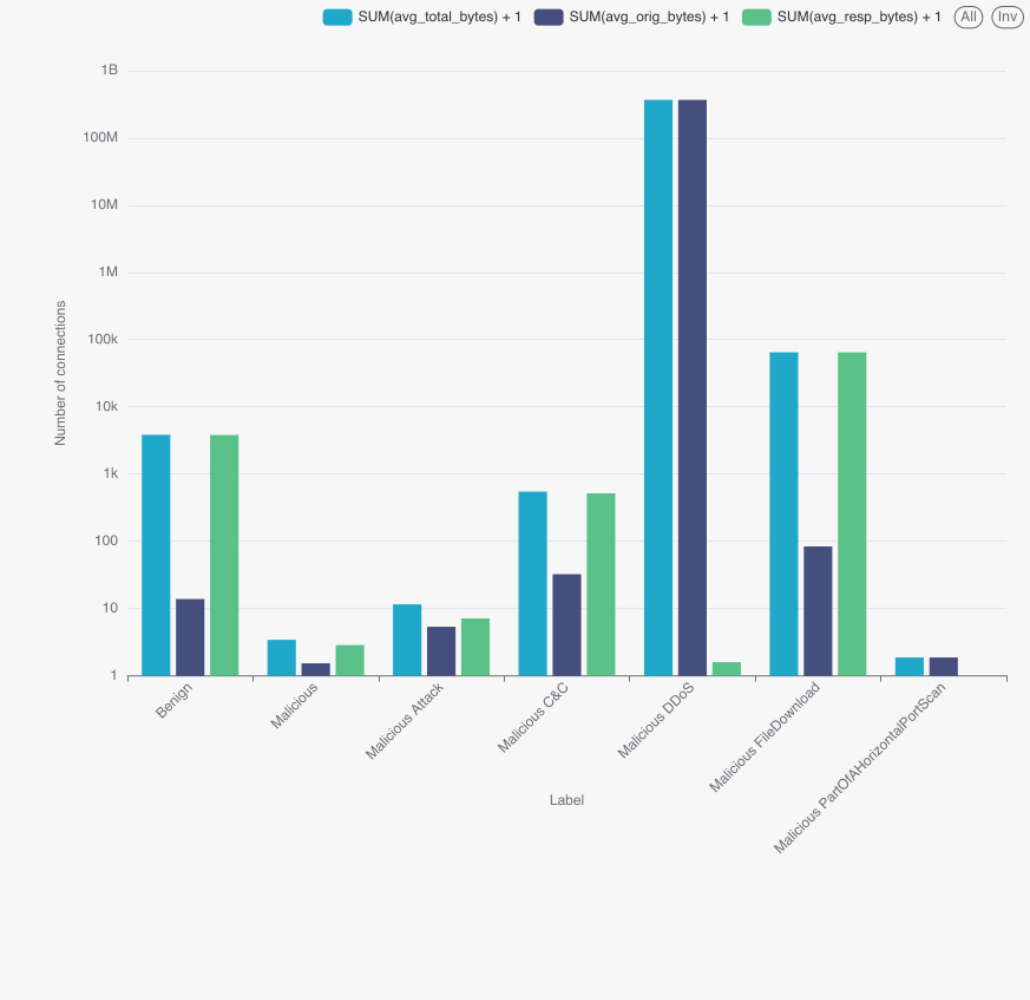

Figure 5: Average originator and responder bytes per connection by label.

Label	Avg orig bytes	Avg resp bytes	Avg total bytes
Benign	12.87	3,830.63	3,843.51
Malicious	0.54	1.87	2.41
Malicious DDoS	3.71×10^8	0.60	3.71×10^8
Malicious C&C	31.48	517.93	549.41
Malicious Attack	4.35	6.16	10.51
Malicious PartOfAHorizontalPortScan	0.87	0.00	0.87
Malicious FileDownload	83.33	64,982.00	65,065.33

Table 7: Average per-connection byte counts by label.

Interpretation. DDoS averages $\sim 3.7 \times 10^8$ originator bytes per flow — six orders of magnitude over benign traffic — because the per-flow payload is dominated by amplified bursts. Port-scan and generic Malicious carry less than 1 byte per connection on average, reflecting a flood of SYN-only probes. Benign traffic has a small originator but a non-trivial responder (the IoT device receives small messages and gets non-trivial server replies). FileDownload, though only 3 rows, shows the expected asymmetric pattern with a heavy responder side.

4.4 Insight 4 — Per-capture maliciousness ratio

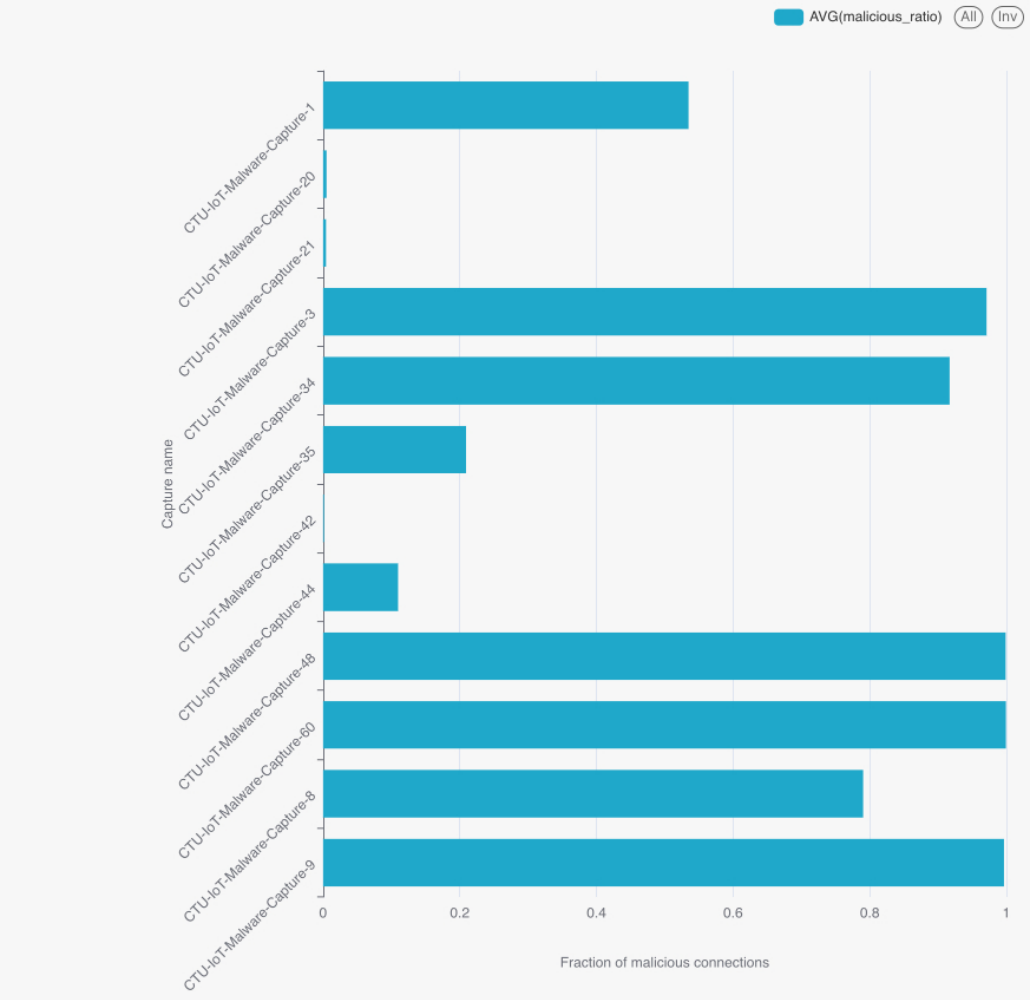

Figure 6: Total flows and malicious ratio per capture.

Interpretation. Capture-level distribution is strongly bimodal: 8 of the 12 captures are $> 79\%$ malicious; 4 captures are $< 11\%$ malicious. There is no middle ground. This means each PCAP was deliberately recorded around a single behaviour. The practical consequence for modelling is that captures should be treated as *strata*: a row-level random split (which we use) does not guarantee unseen captures in the test set, and a stricter group-aware split would be the natural next iteration.

4.5 Insight 5 — Connection state per label

Interpretation. A SYN-only flow (`conn_state = S0`) is the universal signature in this dataset. Even benign IoT traffic is dominated by devices repeatedly trying to reach a server that does not reply. DDoS flows split between `OTH` (no SYN seen, mid-flow capture) and `RSTOS0` (the originator gets a RST after SYN). C&C flows show a characteristic `S3` fingerprint (originator finishes, responder still sending), consistent with long-lived control sessions.

Label	Dominant `conn_state`	Rows	Share within label
Benign	S0	8,722,217	99.3%
Malicious	S0	7,036,199	99.7%
Malicious DDoS	OTH + RSTOS0	5,777,712	99.99%
Malicious C&C	S0 + S3	7,655	88%
Malicious PartOfAHorizontalPortScan	S0	3,386,241	100%

Table 8: Dominant TCP connection state per label.

4.6 Insight 6 — Top targeted ports

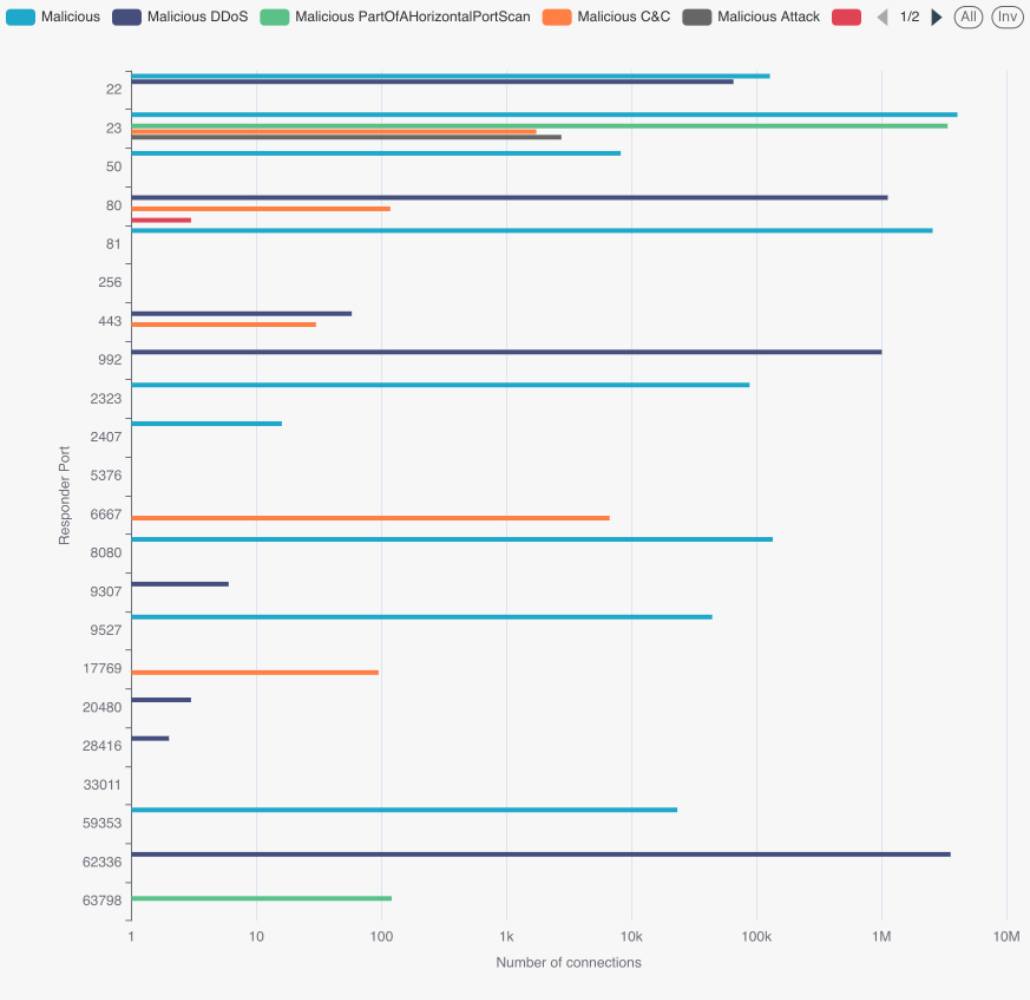

Figure 7: Top targeted destination ports per non-Benign label.

Interpretation. Port 23 (Telnet) is the single most attacked destination: 4M generic Malicious flows and 3.4M port-scan flows target it. This is the classic Mirai-family entry point on unpatched IoT devices. DDoS is bimodal between the high port `62336` (a Mirai-style echo/control port) and HTTP `80`. C&C uses IRC port `6667` heavily.

Label	Port	Flows
Malicious	23	4,055,327
Malicious DDoS	62336	3,578,391
Malicious PartOfAHorizontalPortScan	23	3,386,119
Malicious	81	2,571,979
Malicious DDoS	80	1,125,806
Malicious DDoS	992	1,008,351
Malicious C&C	6667	6,706

Table 9: Top 7 targeted destination ports across non-Benign labels.

4.7 Insight 7 — Average packets per connection

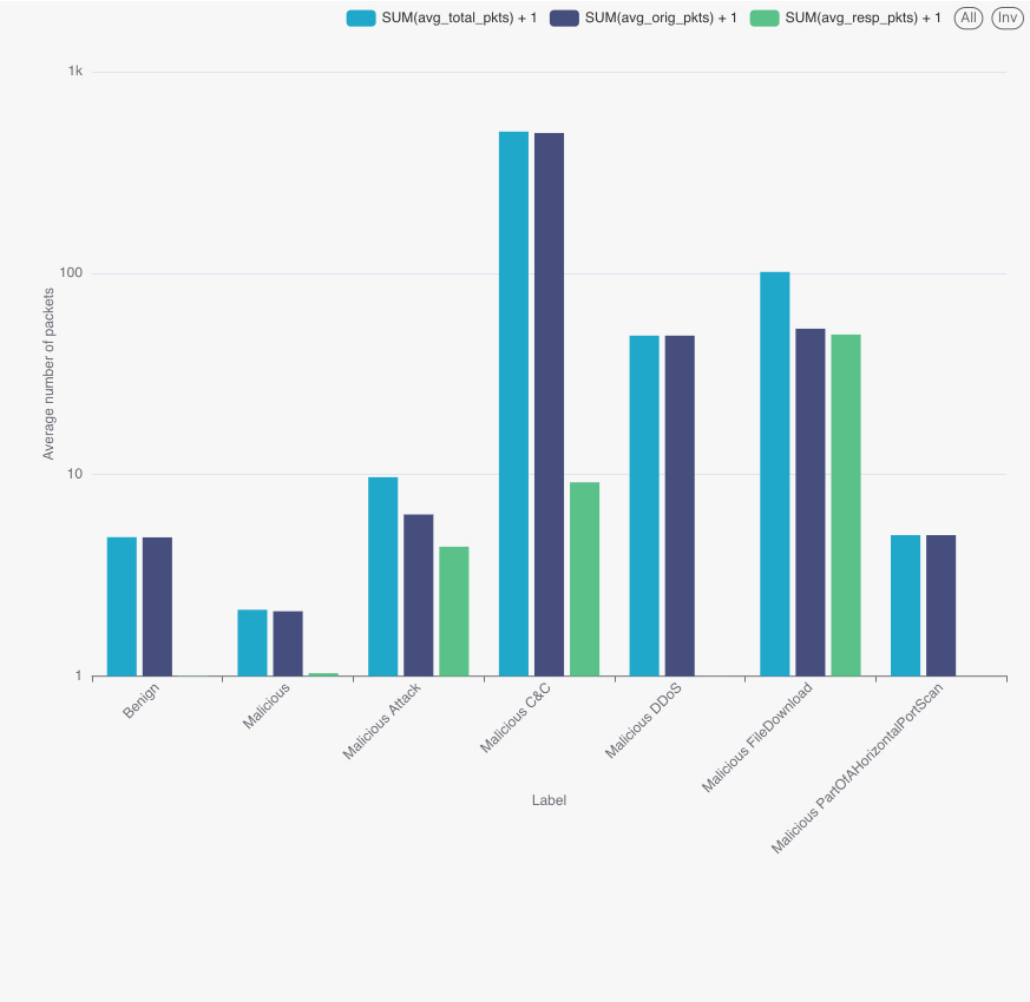

Figure 8: Average originator and responder packets per connection by label.

Interpretation. Long-lived C&C sessions stand out sharply: ∼495 originator packets per session, with a real (8-packet) response. This is very different from one-shot port-scan or flood traffic, where the responder side is essentially zero. The combination of `orig_pkts`, `resp_pkts` and `conn_state` carries most of the discriminative signal that the Random Forest later picks up.

5 ML Modeling

5.1 Feature extraction and preprocessing

`scripts/ml_data_preparation.py` builds a single PySpark `Pipeline` over the Hive partition. The pipeline is fitted only on the training fold to prevent leakage:

1. **Cyclical time encoding.** A custom `CyclicalTimeTransformer` extracts year, month, day, hour, minute, second from `ts` and produces sine/cosine pairs for the five periodic components. This prevents the model from inferring a spurious linear ordering across time wrap-arounds.
2. **Imputation.** `tunnel_parents` is dropped (constant null). Strings are filled with `"unknown"`; numeric nulls become 0; boolean nulls become `False`.
3. **Categorical encoding.** `StringIndexer + OneHotEncoder` for `proto`, `service`, `conn_state`, `history`.
4. **Feature selection.** A `ChiSqSelector(fpr=0.1)` over the categorical block keeps only label-correlated dummies.
5. **Assembly + scaling.** `VectorAssembler` concatenates numeric features, time encodings and the ChiSq-selected categoricals; `StandardScaler(withMean=False, withStd=True)` produces the final `features` column.

The label column is handled by a `StringIndexer` fitted on the same training fold. The split is 60%/40%, seed 42, and the train/test DataFrames are persisted as Parquet on HDFS so that training and evaluation read from a stable on-disk artefact.

5.2 Models and hyperparameter tuning

Both models are wrapped in a Spark `CrossValidator(numFolds=2, parallelism=3)` optimising the multiclass `f1` metric. Each `spark-submit` runs on YARN with 6 executors × 3 cores × 6 GB. The training set is `persist(MEMORY_AND_DISK)` so that CV folds can amortise the I/O.

Model	Grid	Best
Random Forest	numTrees $\in \{30, 60\}$ maxDepth $\in \{4, 6\}$ impurity = gini (default)	numTrees = 60 maxDepth = 6 impurity = gini
Logistic Regression (multinomial)	regParam $\in \{0.01, 0.1\}$ elasticNetParam $\in \{0.0, 0.5\}$ maxIter = 50, family = multinomial	regParam = 0.01 elasticNet = 0.0

Table 10: Hyperparameter grid and the values selected by 2-fold cross-validation on `f1`.

5.3 Evaluation

The evaluation script (`scripts/ml_evaluation.py`) reads the HDFS-persisted predictions and computes four standard multiclass metrics using PySpark `MulticlassClassificationEvaluator`:

Model	Accuracy	Precision	Recall	F1
Random Forest	**0.9979**	**0.9979**	**0.9979**	**0.9978**
Logistic Regression	0.3510	0.3395	0.3510	0.1824

Table 11: Held-out evaluation on ∼10M test rows.

Interpretation. The Random Forest reaches $F_1 = 0.998$ across seven classes — a 5.5× lift over the multinomial linear baseline at the same feature set. The decision boundary on `conn_state` + `id_resp_p` + `orig_pkts` is sharply non-linear after one-hot expansion, which is exactly where bagging and shallow tree splits dominate a globally-linear model. Predictions are persisted to HDFS as CSV so that the evaluator is decoupled from training and can be re-run without retraining.

6 Data Presentation

6.1 Dashboard description

The defence-day dashboard is built in **Apache Superset**, connected to the cluster's Hive 2 endpoint (`jdbc:hive2://hadoop-03.uni.innopolis.ru:10001`). It is a single public dashboard, refreshed on demand against the result tables produced by Stages 2 and 3. Six logical sections were composed:

A. Data characteristics. Headline metrics — 25M rows, 23 features, 7 classes, 12 captures — plus a one-glance snapshot of the pipeline's scope.

B. Class balance (Insight 1). Donut and percentage list of the seven labels, sourced from `q1_results`.

C. Behaviour signatures (Insights 2, 3, 5, 7). Stacked-bar protocol mix, average bytes/-packets per label, and the dominant connection-state per label.

D. Top targets (Insight 6). Port histograms per non-Benign label; the bar for port 23 dominates and is annotated as the Mirai entry point.

E. Capture strata (Insight 4). Per-capture maliciousness ratio, rendered as a bar chart that exposes the bimodal capture-level distribution.

F. Model performance. Comparison panel rendered from `evaluation.csv`, showing accuracy / precision / recall / F1 for both models side by side.

6.2 Findings

The dashboard surfaces three findings that are not obvious from the raw data:

1. **Class skew is non-degenerate.** Four classes carry the bulk of the data and three rare classes carry diagnostic signal. Treating the task as multiclass (and not binary) preserves that diagnostic value.

2. **The attack surface is concentrated.** Port 23 alone accounts for more than 7M malicious flows. A single network policy that rate-limits inbound Telnet on the IoT subnet would have cut the bulk of the malicious volume in this corpus.

3. **A simple model is enough — if non-linear.** A distributed Random Forest with only four hyperparameter combinations explored reaches $F_1 = 0.998$ at this feature set. The same problem is near-unsolvable for a multinomial linear model, which is the strongest practical argument for keeping a tree ensemble in the production classifier.

7 Conclusion

7.1 Summary

This project delivered a fully reproducible end-to-end big-data pipeline that ingests 25M Zeek connection records from the Aposemat IoT-23 corpus, stores them in a partitioned Hive warehouse on HDFS, runs seven HiveQL EDA queries on Tez, trains and cross-validates two distributed Spark ML models on YARN, and presents the results in a public Apache Superset dashboard. The Random Forest model achieves $F_1 = 0.998$ across seven classes on a held-out test set of approximately 10M rows. Every artefact in the `output/` folder is regenerable from a single `main.sh`.

7.2 Reflections on own work

The team's role split worked well: the data engineer owned ingestion and the Hive warehouse layout, the ML specialist owned the Spark pipeline and hyperparameter grid, the data scientist owned EDA and dashboard storytelling, and the tester/technical writer documented every stage and ran the reproducibility checks. The reproducibility contract — "one `main.sh`, every stage idempotent" — was particularly valuable because it forced us to clean up intermediate state at the start of each script rather than relying on a previous run's artefacts.

7.3 Challenges and difficulties

- **Type coercion at ingestion.** The raw Zeek CSVs use sentinel `-` values for missing fields and float-formatted integers (`1234.0`). The PostgreSQL transformation insert had to handle both via `NULLIF` + `REGEXP_REPLACE` before casting — a single missed regex caused a full reload.
- **Hive partitioning vs. skew.** Label-partitioning `network_connections_part` produced one tiny partition (`Malicious FileDownload`, 3 rows) and four very large ones. This is acceptable for analytical scans but costs us shuffle balance in Spark; the workaround was `repartition(18)` after reading.
- **LR convergence.** The multinomial Logistic Regression converged to a near-trivial solution under the given feature set — not a bug, but a structural mismatch with the non-linear decision boundary. Reporting this honestly was important rather than tuning it further.
- **Cluster network access.** Pushing artefacts from `hadoop-01` to GitHub required a small workaround for DNS resolution of `github.com`, solved by an explicit `HostName` entry in `~/.ssh/config`.

7.4 Recommendations

1. Replace the row-level random split with a group-aware split per `capture_id` (Insight 4 motivates this directly).
2. Add Gradient-Boosted Trees to the model grid; they should out-perform Random Forest on this kind of categorical-heavy feature space.
3. Stream a copy of the same Hive partitions into Spark Structured Streaming and serve the trained Random Forest as a near-real-time classifier.
4. Extend the dashboard with per-class precision/recall and a confusion-matrix heatmap — the current panel summarises overall metrics only.

7.5 Table of contributions

Task	Ivan I.	Kirill S.	Nikita Z.	Dmitry T.	Deliverables
Dataset selection & access	25%	25%	25%	25%	IoT-23 captures
Stage 1 — PostgreSQL schema & load	60%	10%	10%	20%	`create_tables.sql`, `build_projectdb.py`
Stage 1 — Sqoop & HDFS warehouse	70%	10%	0%	20%	`import_to_hdfs.sh`
Stage 1 — Storage benchmark	40%	40%	0%	20%	`benchmark_storage_*`, CSVs
Stage 2 — Hive warehouse setup	40%	20%	20%	20%	`db.hql`, partitioned tables
Stage 2 — 7 EDA queries	10%	0%	80%	10%	`q1.hql` …`q7.hql`
Stage 2 — EDA charts & storytelling	10%	0%	80%	10%	`output/q*.jpg`, `q*.csv`
Stage 3 — Feature pipeline	10%	75%	10%	5%	`ml_data_preparation.py`
Stage 3 — Model training & CV	10%	80%	5%	5%	`ml_models_training.py`, `models/`
Stage 3 — Evaluation	10%	60%	20%	10%	`ml_evaluation.py`, `evaluation.csv`
Stage 4 — Superset dashboard	10%	10%	70%	10%	Public dashboard
Pipeline orchestration (`main.sh`)	50%	20%	10%	20%	`main.sh`, `stage*.sh`
Repository documentation	10%	10%	10%	70%	`README.MD`, `docs/`
Final report (this document)	5%	5%	10%	80%	`report/report.pdf`
Defence presentation	10%	20%	30%	40%	`presentation/presentation.pdf`
QA & reproducibility checks	15%	15%	15%	55%	Pipeline re-runs

Table 12: Contribution table. Each row sums to 100% across the four team members.

Repository: github.com/Dmitry057/big-data-big-final-group-case-study-project
Local path on cluster: `/home/team30/project30` (host `hadoop-01.uni.innopolis.ru`, user `team30`)